

TECHNICAL REQUIREMENTS DOCUMENT (TRD)

Health Risk Dashboard

TY B.Sc. Data Science – Mini Project

1. Technical Overview

Build a local web-based Health Risk Dashboard for a TY B.Sc. Data Science mini project. The system will ingest an anonymized healthcare dataset, preprocess the data, visualize health-risk patterns, run a machine-learning classifier, and explain predictions using SHAP.

Primary architecture: Streamlit UI → Python analytics layer → trained ML model → SHAP explanation → dashboard results. CSV will be the initial data source; SQLite may be added if structured storage is required.

2. Recommended Technical Stack

Programming language: Python 3.11+

Frontend/UI: Streamlit

Data processing: Pandas, NumPy

Visualization: Plotly, Matplotlib, Seaborn

Machine learning: Scikit-learn

Primary model: Random Forest Classifier

Explainable AI: SHAP

Database: SQLite (optional for MVP)

Optional local LLM: Qwen3-4B via Ollama

Development: VS Code or Jupyter Notebook

Version control: Git + GitHub

Deployment: Local laptop; optional free hosting for the dashboard where compatible

3. Hardware Requirements

Minimum: Dual-core/quad-core modern CPU, 8 GB RAM, 10 GB free storage.

Recommended: 4+ CPU cores, 16 GB RAM, 20 GB free storage if the local LLM is used.

GPU: Not required for the core dashboard or Random Forest model. A GPU is optional for future deep-learning experiments.

4. Software Requirements

Operating system: Windows, macOS, or Linux.

Python environment: venv or Conda.

Required Python packages: streamlit, pandas, numpy, scikit-learn, shap, plotly, matplotlib, seaborn, joblib.

Optional packages: sqlite3 (Python standard library), requests, openpyxl, and packages required by the selected dataset format.

Optional local LLM runtime: Ollama.

5. System Architecture

Layer 1 – Presentation: Streamlit pages, filters, charts, KPI cards, prediction form, and explanation panels.

Layer 2 – Application: input validation, preprocessing calls, prediction logic, dashboard calculations, and result formatting.

Layer 3 – Analytics: Pandas/NumPy processing, statistical calculations, and feature engineering.

Layer 4 – ML/XAI: Scikit-learn Random Forest model and SHAP explainer.

Layer 5 – Storage: CSV dataset and optional SQLite database; serialized model stored using joblib.

6. Data Requirements

Expected fields may include Patient ID, Age, Gender, BMI, Blood Pressure, Glucose Level, Cholesterol, Smoking Status, Physical Activity, Medical History, and Health Risk/Outcome.

Dataset should preferably contain 1,000–10,000 anonymized records for the MVP.

Data validation shall check required columns, data types, missing values, duplicate records, impossible values, and target-label availability.

No personally identifiable information should be required.

7. Data Processing Pipeline

1. Load CSV data.
2. Validate schema and required columns.
3. Remove duplicate records where appropriate.
4. Handle missing numerical and categorical values.
5. Detect obvious invalid/out-of-range values.
6. Encode categorical variables.
7. Scale features only where required by the selected model; Random Forest does not require feature scaling.
8. Split data into training and testing sets using stratification when appropriate.
9. Save preprocessing/model artifacts for dashboard inference.

8. Machine Learning Requirements

Primary model: Random Forest Classifier.

Baseline models: Logistic Regression and Decision Tree for comparison.

Training: use a reproducible random seed and stratified train/test split or cross-validation.

Class imbalance: inspect class distribution and use suitable class weights/resampling if necessary.

Model artifact: save the final trained pipeline/model using joblib.

Prediction output: predicted class and probability where supported.

9. Explainable AI Requirements

Use SHAP to generate global feature importance and local explanations for individual predictions.

For each prediction, show the top contributing features and their direction of contribution where supported.

Explanations must describe model behavior, not claim medical causation.

SHAP calculations should use the same feature representation passed to the trained model.

10. Dashboard Technical Requirements

Dashboard pages: Overview, Risk Analysis, Prediction, Model Performance, and About/Methodology.

Overview: total records, risk counts, averages, and key charts.

Risk Analysis: interactive filters and comparisons across demographic, clinical, and lifestyle variables.

Prediction: form for patient attributes, validation, prediction result, probability, and SHAP explanation.

Model Performance: confusion matrix, classification report, and selected evaluation metrics.

Use Plotly for interactive charts where practical.

11. API / LLM Requirements

No paid external API is required.

Optional local LLM: Qwen3-4B through Ollama. The LLM may convert already-computed statistics and model explanations into plain-language summaries.

LLM input should be restricted to non-sensitive derived results and should not be allowed to invent clinical conclusions.

If Ollama is unavailable, the dashboard must continue working without the LLM.

12. Security & Privacy

Use only anonymized or synthetic academic data.

Do not store names, phone numbers, addresses, medical record numbers, or other unnecessary identifiers.

Validate uploaded files and restrict accepted file types to expected formats.

Keep secrets and configuration outside source code.

Do not expose patient-level data through public deployment.

13. Performance Requirements

Dataset loading and dashboard initialization should normally complete within a few seconds for an MVP dataset on the target laptop.

Interactive filters should respond without unnecessary model retraining.

The ML model should be trained offline during development; the dashboard should load the saved model for prediction.

SHAP explanations should be generated only for the required prediction/summary rather than recalculating unnecessarily.

14. Project Folder Structure

health-risk-dashboard/

■■■■ app.py

■■■■ data/

■■■■ models/

■■■■ src/

■ ■■■■ preprocessing.py

■ ■■■■ model.py

■ ■■■■ explainability.py

■ ■■■■ analytics.py

■■■■ notebooks/

■■■■ requirements.txt

■■■■ README.md

■■■■ .gitignore

15. Testing Requirements

Unit tests: preprocessing functions, validation, feature transformations, and prediction functions.

Functional tests: dataset upload, filters, dashboard navigation, prediction form, and explanation display.

Model tests: verify expected input shape, saved model loading, and metric calculation.

Error tests: invalid CSV, missing columns, empty file, missing required input, and unsupported values.

User acceptance: evaluator can complete the main workflow without technical assistance.

16. Deployment Requirements

Primary deployment: run locally using Streamlit.

Typical startup command: streamlit run app.py

Provide requirements.txt for reproducible installation.

Optional public deployment should use only non-sensitive/anonymized data and should not require paid services.

17. Technical Acceptance Criteria

The application launches successfully on the target laptop.

A valid healthcare CSV can be loaded and validated.

The dashboard displays correct KPIs and interactive visualizations.

The saved ML model produces a prediction for valid patient input.

SHAP or equivalent feature explanations are displayed for predictions.

Model evaluation metrics are available.

The project runs without paid APIs or proprietary model dependencies.

The GitHub repository contains source code, setup instructions, dataset documentation, and project limitations.